ChatGPT

Comparative Deep Research on Claude Code, OpenAI Codex, Gemini CLI, and Google Antigravity

Executive summary

Across the four documentation sites, a clear convergence is visible: modern coding agents are being productized as **local-first development companions** that can (a) read and modify files, (b) run shell commands, and (c) extend their capabilities via **standardized "skills" packages** and **tool-integration protocols**—with increasingly formalized controls for approvals, sandboxing, and enterprise governance. Claude Code, OpenAI Codex, and Gemini CLI all explicitly align their "skill" systems with the **Agent Skills open standard** (agentskills.io) and treat skills as **on-demand, token-efficient expertise modules** rather than always-on prompt stuffing. [1]

The most practically important differentiators are **(1) the extensibility surface** (plugins/extensions vs config-only), **(2) the control plane for safety** (permission rules, sandbox modes, hook interception), and **(3) the commercial/auth split** between consumer subscriptions and API-billed usage. Claude Code offers a strongly-scoped configuration model (managed/user/project/local), a first-class plugin system, a tiered permissions model (including "Plan Mode" and explicit bypass warnings), and deep MCP integration. [2] OpenAI Codex's docs emphasize multi-surface usage (app, IDE extension, CLI), a ChatGPT-vs-API-key authentication fork (with feature differences like "fast mode"), and configuration via `~/.codex/config.toml` plus command-line overrides. [3] Gemini CLI is the most thoroughly documented as a **scriptable** agent runtime: it provides a headless mode with JSON/JSONL event streams, multiple sandboxing backends, hooks with strict JSON I/O and structured exit-code semantics, and a deep MCP implementation reference (including OAuth token storage and tool filtering). [4]

Google Antigravity appears positioned as an **agent-first IDE** ("Mission Control" for multiple agents) with explicit policies for terminal execution, artifact review frequency, and browser JavaScript execution risk, plus a tightly integrated Chrome "browser subagent" for web-automation and validation. [5] However, Antigravity's primary documentation pages (`antigravity.google/docs/...`) were not directly retrievable in full (likely client-rendered); this report therefore relies on Google's Codelabs content and search-indexed excerpts from the official docs, and marks many specifics as **unspecified** where the source text could not be fully accessed. [6]

A robust cross-platform strategy emerges: design your coding agent around a **canonical "Skill Pack" format** (Agent Skills) paired with **MCP-based tool connectors**, wrap execution in a **policy + sandbox + hook interception layer**, and implement "surface adapters" for Claude Code plugins, Codex config/rules, and Gemini CLI settings/extensions. This report provides concrete mapping tables, recommended design patterns, sample definitions (YAML/JSON), and implementation/migration checklists.

Scope, methodology, and source limitations

This report is based on the four user-provided documentation roots and their discoverable subpages, with the research date anchored to **March 15, 2026 (Asia/Phnom_Penh)**. For each platform, information is drawn from the official docs pages that were accessible to retrieval tools, and all key claims are cited.

A key limitation affected only **Google Antigravity**: multiple `antigravity.google` pages returned no parseable content when opened directly, even though they are indexed and snippets are available. As a result, Antigravity coverage is derived from the **official Google Codelab** and search-indexed excerpts of Antigravity documentation pages, and details not visible there are explicitly labeled **unspecified**. [7]

For the other three platforms (Claude Code, OpenAI Codex, Gemini CLI), subpages were accessible and provided sufficient primary-source detail for implementation-oriented guidance, including configuration keys, file locations, tool semantics, hook schemas, and concrete command examples. [8]

Platform deep dives

Claude Code

Positioning and core capabilities

Claude Code is a local terminal-based coding agent designed to work inside your repositories with file reads/edits, shell execution, and extensibility via skills, subagents, plugins, hooks, and MCP servers. The docs emphasize agentic workflows integrated with typical development tasks (editing files, running tests, working with git), plus enterprise-grade administration and managed policies. [9]

Claude Code's operational model includes a strong separation between: - **Persistent project guidance** (CLAUDE.md and rules) loaded into context at session start or on-demand by path. - **On-demand procedural capabilities** (skills). - **Delegated, scoped work** (subagents; and separately "agent teams" for multi-session orchestration). [10]

Installation and CLI surface

Claude Code is invoked as `claude` and supports numerous CLI options (including specifying working directories and adding additional directories for access), plus a broad collection of interactive slash commands. The CLI reference documents the `claude` command structure and subcommands, and indicates that you can inspect versions via `claude --version`. [11]

A distinctive aspect of Claude Code is that "configuration" is treated as a **multi-scope system** (managed/user/project/local), with settings editable through the `/config` UI inside the REPL. [12]

Persistent project context: CLAUDE.md and rules

Claude Code offers two complementary "memory" mechanisms across sessions:
- **CLAUDE.md files** (instructions you write)
- **Auto memory** (notes Claude writes from your corrections/preferences) [13]

Key properties from the docs: - CLAUDE.md files can exist at **organization-managed**, **project**, and **user** scopes, each with different paths and precedence (e.g., `./CLAUDE.md` or `./.claude/CLAUDE.md` for project scope; `~/.claude/CLAUDE.md` for user scope). [13] - CLAUDE.md supports importing other files via an `@path/to/import` syntax with a maximum recursion depth of five hops, and user approval is requested the first time external imports are encountered. [13] - For large repos, `.claude/rules/` supports modular instructions with optional YAML frontmatter `paths:` to scope rules to matching globs (e.g., only `src/api/**/*.ts`). [13] - `/init` can generate a starter CLAUDE.md by analyzing your codebase and discovering build/test conventions. [13]

Implication for coding-agent builders: Use CLAUDE.md and rules for **stable, always-relevant constraints** (security policy, build/test commands, architectural rules), and reserve skills for **task-shaped procedures** that shouldn't consume context every session. [14]

Skills as reusable procedures

Claude Code "skills" are created via `SKILL.md` and expose slash commands (legacy `.claude/commands/*.md` works, but "custom commands have been merged into skills"). [15]

Notable documentation points: - Skills live under `.claude/skills/<skill-name>/SKILL.md` (with optional supporting files), and Claude Code can discover skills from nested directories in monorepos (e.g., `packages/frontend/.claude/skills/`). [16] - Skills follow the **Agent Skills open standard** and include frontmatter that can control invocation behavior (e.g., `disable-model-invocation: true` prevents the model from running the skill implicitly). [17] - Skill descriptions can be loaded "lightweight" into context for discoverability, while full skill content is loaded only upon invocation; subagents may preload full skill content at startup. [18]

Sample Claude Code skill (SKILL.md)

```
---
name: deploy
description: Deploy the application to production.
disable-model-invocation: true
---

Deploy $ARGUMENTS to production:

1. Run the test suite
2. Build the application
3. Push to the deployment target
4. Verify the deployment succeeded
```

This invocation-control behavior is explicitly documented: `disable-model-invocation: true` removes the description from context and blocks implicit model invocation until you run it. [19]

Subagents and agent teams

Claude Code supports **subagents**: specialized assistants that run in their own context window with a custom prompt, constrained tool access, and independent permissions. Subagents can preserve main-session context and enforce constraints, and the docs explicitly recommend routing some workloads to cheaper/faster models for cost control. [20]

Management options include: - `/agents` interactive UI for creating/editing/removing subagents, listing sources, and seeing overrides; plus `claude agents` to list from the command line. [21]

Claude Code also documents **agent teams** (experimental): multiple Claude Code instances coordinated by a team lead, enabled via an experimental setting (`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS`) and requiring Claude Code v2.1.32+. [22]

Design takeaway: Use subagents for **specialized "internal roles"** (e.g., security review, test triage, performance profiling) where the main session should remain clean; consider agent teams only when you need **parallel sessions with direct teammate interaction**. [23]

Hooks, plugins, and extensibility

Claude Code supports a plugin system with a manifest in `.claude-plugin/plugin.json` plus standard root directories for commands, agents, skills, and hooks. The plugins reference specifies that component directories must be at the plugin root (not inside `.claude-plugin/`). [24]

The plugins reference includes: - Default structure and component locations (e.g., `commands/`, `agents/`, `skills/`, `hooks/hooks.json`, `.mcp.json`, `.lsp.json`). [25] - Validation and troubleshooting guidance, including `claude plugin validate` and hook event-name correctness (e.g., `PostToolUse`, not `postToolUse`). [25]

Claude Code's "hooks" are an event-driven automation mechanism (command/prompt/agent hook types are referenced in troubleshooting), enabling enforcement and automation around tool usage. [26]

MCP tool integration

Claude Code supports the **Model Context Protocol (MCP)** and positions it as a way to connect "hundreds of external tools and data sources" via an open standard. [27]

Key operational details: - Project-scoped MCP servers are stored in a `.mcp.json` at project root and are intended for version control; an example `claude mcp add --transport http ... --scope project ...` is documented. [28] - For security, Claude Code prompts for approval before using project-scoped servers from `.mcp.json`, and provides `claude mcp reset-project-choices` to reset approvals. [28] - User-scoped servers are stored in `~/.claude.json`. [29]

Sample `.mcp.json` structure (project scope)

```
{
  "mcpServers": {
    "shared-server": {
      "command": "/path/to/server",
      "args": [],
      "env": {}
    }
  }
}
```

This standardized format is shown directly in the Claude Code MCP documentation. [28]

Authentication, governance, and permissions

Claude Code supports multiple authentication methods: individual Claude.ai logins (Pro/Max), Teams/Enterprise accounts, Claude Console accounts, and cloud-provider authentication (Amazon Bedrock, Google Vertex AI, Microsoft Foundry) via environment variables. [30]

Credential management details include: - On macOS, credentials are stored in encrypted macOS Keychain. - Supported auth types include Claude.ai credentials and multiple provider auth types. - `apiKeyHelper` can run a shell script to return an API key; default refresh behavior is after 5 minutes or on HTTP 401; TTL can be customized via `CLAUDE_CODE_API_KEY_HELPER_TTL_MS`. [31]

Claude Code has a **tiered permission system** and multiple permission modes: - Tool categories differentiate read-only operations vs bash vs file modification. - `/permissions` UI manages allow/ask/deny rules, evaluated deny → ask → allow (first match wins). [32] - Modes include `default`, `acceptEdits`, `plan` (read-only), `dontAsk` (auto-deny unless preapproved), and `bypassPermissions` (explicitly warned as only for isolated environments; can be disabled by managed settings). [32]

Sample permissions snippet (blocking `.env` reads and `curl`, allowing specific npm commands) is shown in the settings documentation. [33]

Rate limits, APIs, pricing

Claude Code's documentation focuses on product usage and enterprise administration rather than publishing numeric rate limits within the pages accessed for this research. Subscription references appear in authentication guidance (e.g., Claude Pro/Max and Teams/Enterprise), but **explicit rate limits/pricing for Claude Code itself are not specified in the accessed sources**. [30]

OpenAI Codex

Positioning and surfaces

OpenAI Codex is documented as a multi-surface coding agent usable via: - Codex app - IDE extension - Codex CLI [34]

Codex CLI is described as a local terminal coding agent that can read, change, and run code in a selected directory; it is also described as open source and built in Rust. [35]

Codex's docs consistently emphasize a **commercial/authentication split**: - ChatGPT subscription-based access (with some credit-tied features) - API-key access billed through the OpenAI Platform at standard API rates [36]

Installation and CLI basics

The Quickstart documents installing Codex CLI and provides both npm and Homebrew installation commands, then running `codex` and signing in, and then asking it to work in the current directory. [37]

Documented commands include:

```
npm install -g @openai/codex
brew install codex
codex
```

[37]

The Codex CLI page further documents upgrading via npm (`npm i -g @openai/codex@latest`). [38]

Platform support is described with nuance: the Codex CLI is "available on macOS and Linux," while Windows support is "experimental," with guidance that WSL workspaces provide the best Windows experience. [38]

Agent mode and approvals

The Quickstart explicitly states that Codex "starts in Agent mode by default," where it can read files, run commands, and write changes in your project directory. [37]

The IDE extension features page excerpt notes that increasing reasoning effort can consume more tokens and "consume your rate limits faster," and that users can choose reasoning effort levels. It also refers to approval modes and notes that Agent mode is the default. [39]

Unspecified: The complete set of approval-policy keys and their exact semantics are likely documented under "Agent approvals & security," but those pages were not retrieved in full in this session.

Authentication and credential storage

Codex supports signing in with an API key across app/CLI/IDE extension. Key points from the authentication doc: - API key usage is billed through your OpenAI Platform account at standard API rates. [40] - Some features (example: "fast mode") rely on ChatGPT credits and are available only when signed in with ChatGPT rather than API key. [40] - Login caching: Codex caches login details and reuses them; CLI and extension share the cached login. [41] - Storage: cached login details are stored locally in plaintext at `~/.codex/auth.json` **or** in an OS-specific credential store. [41] - If an account supports multiple login methods and one is email/password, MFA is required before accessing Codex. [41]

Security implication: plaintext storage is a high-risk default in some environments; enterprises should harden file permissions, prefer OS credential stores when possible, and restrict runner images where `~/.codex/auth.json` may persist. [41]

Configuration system

The Codex CLI reference excerpt states: - The CLI inherits most defaults from `~/.codex/config.toml`. - Command-line `-c key=value` overrides take precedence for that invocation. [42]

The Config Reference excerpt clarifies: - User-level configuration lives in `~/.codex/config.toml`. - Project-scoped overrides can live in `.codex/config.toml` and are loaded only when you trust the project. [43]

A "Sample Configuration" page is described as a complete example `config.toml` you can copy and adapt. [44]

Practical pattern: For a cross-platform stack, treat `~/.codex/config.toml` as the equivalent of Claude Code's user scope and Gemini CLI's `~/.gemini/settings.json`, and `.codex/config.toml` as a project scope gated behind a trust model. [45]

Skills system

Codex documents skills as an on-demand mechanism that the agent can activate explicitly or implicitly: - Explicit: include the skill in the prompt; in CLI/IDE, use `/skills` or type `$` to mention a skill. - Implicit: Codex chooses a skill when the task matches the skill description; descriptions should be written with clear boundaries. [46]

Skill creation guidance includes: - Use a built-in creator (`$skill-creator`) that asks what the skill does, when it should trigger, and whether it should remain instruction-only or include scripts (instruction-only default). [47] - Manual creation: a folder containing `SKILL.md` with YAML frontmatter including `name` and `description`. [48] - Enable/disable: `[[skills.config]]` entries in `~/.codex/config.toml` can disable a skill by path without deleting it. [47]

Example documented fragment:

```toml
[[skills.config]]
path = "/path/to/skill/SKILL.md"
enabled = false
```

[47]

Codex also packages skill assets beyond the SKILL.md (scripts, references, templates) and mentions an `openai.yaml` as an optional component controlling appearance/dependencies, but the full schema and resolution rules for that file were not retrieved in full in this session. [46]

Multi-agent workflows

Codex documents "Multi-agents" as experimental: - Must be explicitly enabled. - Can be enabled via CLI `/experimental` (then restart). - Can also be enabled by setting `[features] multi_agent = true` in `~/.codex/config.toml`. - Current visibility is CLI-first; other surfaces are "coming soon." [49]

Rules for command execution and safety

Codex documents a "Rules" system that can validate and constrain commands: - `match` and `not_match` examples are used to validate rules when loaded. - When Codex considers a command, it compares the command's **argument list** to a `pattern`, treating the command internally as an argv list like `execvp(3)` receives. [50]

This argv-level framing is a crucial security insight: it suggests rules are designed around **structured command arguments**, reducing ambiguity from shell parsing—though the docs also warn about wrappers and compound commands (from the Rules page excerpt). [51]

MCP, SDKs, and APIs

Codex supports the Model Context Protocol (MCP) in both the CLI and IDE extension, with the stated goal of connecting models to third-party tools and context (docs cite examples like browser/Figma). [52]

Codex also documents an "App Server" with endpoints related to thread and turn lifecycle (e.g., `turn/start`, `turn/steer`, `thread/rollback`), and reports three authentication modes with an "API key" mode. [53] This surface is relevant to teams building internal orchestration around Codex sessions.

The Codex changelog notes: - "GPT-5-Codex is now available in the Responses API" and can be used with an API key in Codex CLI. - The model snapshot is expected to update regularly. - It is "available at the same price as GPT-5." [54]

Unspecified: The full Codex SDK API and its code samples were not retrieved in full here; the MCP Server and GitHub Action docs are referenced in navigation but not accessed in full.

Pricing, quotas, and rate limits

Codex Pricing excerpt states: - Codex is included in ChatGPT Plus, Pro, Business, Edu, or Enterprise. - For a limited time, Codex is available for free in ChatGPT Free and Go. - Plus/Pro/Business/Enterprise may receive "2x Codex rate limits." [55]

Unspecified: Numeric rate limits by plan were not visible in the retrieved excerpted content.

Automation and headless use

Codex documents "Non-interactive Mode" as the solution for CI/CD or automation scenarios, especially if a team must run using a Codex user account rather than an API key; it also states that API keys are the right default for automation because they are easier to provision and rotate. [56]

Gemini CLI

Positioning and core interaction model

Gemini CLI is a command-line AI assistant for code and terminal workflows, installed via npm and authenticated via Google account, Gemini API key, or Vertex AI—then extended with tools, skills, hooks, MCP servers, and extensions. [57]

Gemini CLI's docs are unusually explicit about: - **Programmatic/headless execution** (non-TTY / `-p` prompt flag) and structured outputs, - **Sandboxing options and precedence rules** across OS/container backends, - **Hook I/O contracts** with strict JSON parsing expectations and fail-safe behaviors, - **MCP architecture internals** (discovery and execution layers, transport mechanisms). [58]

Installation, execution, and sandbox entrypoints

Gemini CLI installation options include: - npm: `npm install -g @google/gemini-cli` - Homebrew: `brew install gemini-cli` - MacPorts: `sudo port install gemini-cli` - Anaconda: install Node.js in an env then npm-install inside it [59]

It can also run without installation: - `npx @google/gemini-cli` or even directly from GitHub via `npx https://github.com/google-gemini/gemini-cli` [60]

Containerized execution is treated as a core security pathway: - You can run the published sandbox image directly with Docker. - Or run a locally installed CLI inside the sandbox using `--sandbox`, e.g. `gemini --sandbox -y -p "your prompt here"`. [61]

Authentication and credential strategy

Gemini CLI documents multiple authentication methods and the decision logic by user scenario (individual Google accounts, workspace accounts, API keys, Vertex AI, headless mode). [62]

Key documented mechanisms: - **Sign in with Google** via browser; credentials cached locally. [63] - **Gemini API key**: set `GEMINI_API_KEY` in environment; warning to treat API keys as sensitive. [64] - **Vertex AI**: requires `GOOGLE_CLOUD_PROJECT` and `GOOGLE_CLOUD_LOCATION`; supports ADC (`gcloud auth application-default login`), service account JSON key via `GOOGLE_APPLICATION_CREDENTIALS`, or Google Cloud API key. [65] - Persistence options: - Shell config files (with warning that exporting keys makes them readable by any process launched from that shell), - Or `.gemini/.env` files searched upward from current directory and then user home directory (`~/.gemini/.env`). [66]

Context and persistent instructions: GEMINI.md

Gemini CLI uses `GEMINI.md` as its default context file system, with hierarchical discovery: - Global: `~/.gemini/GEMINI.md` - Environment/workspace context: search for `GEMINI.md` in configured workspace directories and parents - "Just-in-time" context discovery when tools access a directory, scanning ancestors up to trusted root [67]

It provides `/memory` subcommands for inspecting, reloading, and appending to global memory, and supports modular imports via `@file.md` syntax. [68]

Tools model and execution semantics

Gemini CLI documents that tools are usually invoked automatically, but can be manually triggered: - `@path` triggers file reads (`read_many_files`) - `!cmd` triggers shell execution (`run_shell_command`) [69]

Security behavior includes: - Mutating tools (file writes/edits, shell commands) require manual approval; the CLI shows diffs or the exact command. [70] - Sandboxing and trusted folders affect whether tools can execute outside safe boundaries. [71] - The `web_fetch` tool includes a documented warning: it can access local/private network addresses (e.g., localhost), posing security risk under untrusted prompts. [72]

The tools reference provides a detailed catalog with parameters for each tool (shell exec, glob/grep/list/read/write/replace, planning tools like `enter_plan_mode` / `exit_plan_mode`, web search/fetch, skills activation, etc.). [73]

Skills: Agent Skills open standard

Gemini CLI's Agent Skills system is anchored in the Agent Skills open standard and designed for progressive disclosure: - Only metadata is injected initially; full instructions load upon activation with user consent. - Skills discovered from workspace (`.gemini/skills/` or `.agents/skills/`), user (`~/.gemini/skills/` or `~/.agents/skills/`), and extension bundles, with precedence Workspace > User > Extension; `.agents/skills/` alias overrides `.gemini/skills/` within a tier. [74] - Skills are activated via the `activate_skill` tool; on activation, the skill directory becomes allowed file paths for reading bundled assets. [75]

Hooks: lifecycle interception as programmable policy

Gemini CLI hooks are synchronous scripts executed at defined lifecycle events, used to inject context, validate actions, enforce policies, and log activity. [76]

Key technical specifics: - Events include `SessionStart`, `BeforeAgent`, `BeforeTool`, `BeforeModel`, `AfterTool`, etc., with impacts like "Block Tool / Rewrite" and "Filter Tools." [77] - **Strict I/O contract** ("Golden Rule"): hooks must output only a final JSON object to stdout; any non-JSON stdout breaks parsing; stderr is for debugging logs. [78] - Exit codes have defined semantics (0 = success with parseable JSON; 2 = critical block; others = warning). [78] - Configuration lives in `settings.json` and merges across layers: project `.gemini/settings.json`, user `~/.gemini/settings.json`, system `/etc/gemini-cli/settings.json`, and extensions. [79] - Hooks execute with a sanitized environment that includes `GEMINI_PROJECT_DIR`, `GEMINI_SESSION_ID`, `GEMINI_CWD`, and also a compatibility alias `CLAUDE_PROJECT_DIR`. [80] - Security warning: hooks execute arbitrary code with user privileges; project-level hooks are fingerprinted and changes produce warnings. [80]

MCP servers: deep integration reference

Gemini CLI contains an unusually deep technical reference explaining how MCP tools are discovered and executed: - Discovery orchestrated from configured `mcpServers` in `settings.json`, supporting transports like Stdio, SSE, and Streamable HTTP, with schema sanitization for Gemini compatibility. [81] - Configuration supports properties like `command` / `args` (Stdio), `url` (SSE), `httpUrl` (HTTP streaming), plus env var expansion and timeouts. [82] - OAuth support is deeply documented: `/mcp auth` management, token storage in `~/.gemini/mcp-oauth-tokens.json`, refresh behavior, and auth provider types like `dynamic_discovery` and `google_credentials`. [83] - Example configs show Docker-based servers, HTTP servers with headers, tool filtering via `includeTools` / `excludeTools`, and service-account impersonation for protected endpoints. [84]

The MCP setup tutorial illustrates a concrete GitHub MCP server configuration using Docker and environment variable mapping to avoid hardcoding secrets in config. [85]

Headless mode for automation

Headless mode is triggered automatically in non-TTY environments or via `-p/--prompt`. Output formats include: - JSON output with `response`, `stats`, optional `error`. - Streaming JSONL output with event types like `init`, `message`, `tool_use`, `tool_result`, `error`, `result`. Exit codes include 0 success, 1 general error, 42 input error, 53 turn limit exceeded. [86]

This is a strong foundation for building CI/CD pipelines or "agent-as-a-step" workflows that need stable machine-readable outputs.

Sandboxing

Gemini CLI sandboxing is documented across multiple mechanisms: - macOS Seatbelt (`sandbox-exec`) with built-in profiles and network variants, - Container-based (Docker/Podman), - gVisor/runsc, - LXC/LXD (experimental, full-system container). [87]

Enablement precedence is explicitly documented: 1) `-s/--sandbox` flag
2) `GEMINI_SANDBOX` environment variable
3) `"tools": { "sandbox": ... }` in `settings.json` [88]

This “precedence transparency” is important for enterprise reproducibility: you can enforce sandbox in settings, but allow emergency overrides via flags (or vice versa).

Pricing, quotas, and rate limits

Gemini CLI’s quotas and pricing are unusually explicit and include numeric request limits by auth mode: - Free tier (Google login / Gemini Code Assist for individuals): **1000 model requests/user/day** and **60 requests/user/minute**. [89] - Free tier (Gemini API key unpaid): **250 requests/user/day** and **10 requests/user/minute**, “Flash model only.” [89] - Workspace/organization tiers (license seat-based) include higher daily/minute limits (e.g., Standard: 1500/day; Enterprise: 2000/day; both 120/min). [90] - “Pay as you go” options via Vertex AI or Gemini API key. [90]

Extensions

Gemini CLI extensions package prompts, MCP servers, custom commands, themes, hooks, sub-agents, and agent skills into a shareable format. [91]

Key behaviors: - Manage via `/extensions list` (interactive) and `gemini extensions` commands (terminal). [92] - Install from GitHub URL or local path; `--consent` can acknowledge risks and skip confirmation prompt. [93] - Build via templates (`gemini extensions new my-first-extension mcp-server`) and a `gemini-extension.json` manifest that can define MCP servers. [94] - Best practices include least privilege, `excludeTools` patterns, input validation, and “sensitive settings” stored in system keychain with `sensitive: true`. [95]

Observability

Gemini CLI includes built-in OpenTelemetry support with settings and environment variable overrides (e.g., `GEMINI_TELEMETRY_ENABLED`, `GEMINI_TELEMETRY_TARGET`, endpoint/protocol settings, and `logPrompts` defaulting to true). [96] It also documents logging of security-related events (e.g., hook calls, tool output masking) and detailed semantic conventions. [97]

Google Antigravity

Positioning and surfaces

Google Antigravity is described (in official Google Codelabs) as an “agentic development platform” and an “agent-first” IDE experience that shifts focus from text editing to orchestrating autonomous agents. It presents a “Mission Control” Agent Manager alongside a more traditional editor view, and supports managing multiple agents asynchronously. [98]

The official Antigravity documentation home excerpt describes Antigravity as evolving the IDE into the agent-first era, enabling higher-level, task-oriented agent management across workspaces while retaining an AI IDE core. [99]

Installation and onboarding

The Antigravity "Getting Started" docs excerpt (search-indexed) provides OS requirements and directs users to download from an Antigravity download page. It includes platform constraints (e.g., macOS minimum version and lack of x86 support). [100]

The Google Codelab adds: - Antigravity is in preview and available for personal Gmail accounts. - Requires local install; available on Mac, Windows, and specific Linux distributions. - Requires Chrome and a personal Gmail account for preview. [98]

Antigravity also offers a CLI entrypoint: install a command-line tool to open Antigravity with `agy`. [98]

Agent runtime model

The Antigravity Agent is described in docs excerpt as: - A multi-step reasoning system powered by a frontier LLM - Can reason over existing code - Uses a wide range of tools including the browser - Communicates via tasks and artifacts [101]

The Codelab explains two key "modes": - **Planning** mode: plans, organizes in task groups, produces artifacts; for deep research/complex tasks. - **Fast** mode: executes directly for simple tasks like renaming variables or running a few commands. [98]

Policies, approvals, and security posture

Antigravity's onboarding flow includes explicit policy choices: - Terminal execution policy (auto-proceed vs request review) - Review policy (always proceed vs agent decides vs request review) - JavaScript execution policy for browser tools (always proceed vs request review vs disabled), explicitly noting that always proceeding has the highest exposure to security exploits. [98]

It also describes "Secure mode," "Review-driven development (recommended)," "Agent-driven development," and "Custom configuration" as presets. [98]

The Antigravity Rules/Workflows docs excerpt describes "Rules" as manually defined constraints at local and global levels, configured via a "Customizations" panel. [102]

Browser integration via subagent

Antigravity's browser feature is described in both the Codelab and the docs excerpt: - Antigravity can open, read, and control a Chrome browser to test websites, read sources, and automate tasks. - A browser subagent operates in a separate browser profile to isolate normal browsing. - The subagent can record actions and present screenshots/videos as artifacts. [103]

Skills and reusable workflows

Antigravity's skill system is described in an official docs excerpt: - Workspace skills: `<workspace-root>/.agents/skills/<skill-folder>/` - Global skills:

`~/.gemini/antigravity/skills/<skill-folder>/` - Notes a default shift to `.agents/skills` while maintaining backwards support for `.agent/skills`. [104]

Unspecified: The full Antigravity skill file schema, invocation syntax, and any packaging/marketplace model were not retrievable in full.

Pricing and rate limits

An official Antigravity blog snippet (search-indexed) indicates: - Public preview for individuals at no charge (at the time of writing) - Access is capacity-based with rate limits refreshed every five hours, correlated with "amount of work done by the agent" (not simply prompt count). [105]

Unspecified: Exact numeric rate limits and pricing tiers were not retrievable in full from the primary docs pages during this session.

Cross-platform feature matrix and interoperability mapping

Feature matrix across platforms

The table below summarizes major attributes explicitly documented in sources. "Unspecified" indicates the relevant detail was not visible in the retrieved content.

Attribute	Claude Code	OpenAI Codex	Gemini CLI	Google Antigravity
Primary surface	Terminal agent (`claude`) with integrations, plugins	App + IDE extension + CLI (`codex`)	Terminal agent (`gemini`) with rich scripting	Agent-first IDE ("Agent Manager" + Editor), CLI `agy`
Skills standard	Agent Skills open standard; `SKILL.md` with invocation controls	Agent Skills open standard; `SKILL.md` + creator; enable/disable in `config.toml`	Agent Skills open standard; progressive disclosure; `activate_skill`	Skills mentioned; directories documented; schema unspecified
Skill discovery locations	`.claude/skills/` + nested discovery; legacy `.claude/commands/`	Path-based; enable/disable via `[[skills.config]]` entries; base path unspecified	Workspace `.gemini/skills` or `.agents/skills`, user `~/.gemini/skills` or `~/.agents/skills`, extension skills	Workspace `.agents/skills`; global `~/.gemini/antigravity/skills`

Attribute	Claude Code	OpenAI Codex	Gemini CLI	Google Antigravity
Hooks / interceptors	Hooks system (event-based) packaged via plugins; details partially accessed	Unspecified (approval & security docs exist, not retrieved)	Hooks with strict JSON stdout contract, exit codes, lifecycle events	Policies for terminal/ review/JS; “Rules” panel; hook system unspecified
MCP support	Yes; project `.mcp.json`, user `~/.claude.json`	Yes; MCP in CLI + IDE extension	Yes; deep MCP architecture; OAuth; filtering	Unspecified from retrievable sources
Subagents / multi-agent	Subagents; “agent teams” experimental	Multi-agents experimental feature flag	Subagents (experimental feature flag required for `/agents`)	Multiple agents managed in Agent Manager; subagent browser agent described
Sandboxing and approvals	Permissions system (allow/ask/ deny, modes incl plan/bypass warnings); sandbox settings mentioned	Agent mode; approval modes; reasoning effort impacts rates; details beyond excerpts unspecified	Multi-backend sandboxing with clear precedence; mutator approvals	Setup policies for terminal/JS and review; secure mode
Headless / noninteractive	Unspecified in retrieved Claude Code docs	Non-interactive mode documented	Headless mode with JSON/JSONL output & exit codes	Unspecified
Auth modes	Claude.ai, Teams/ Enterprise, Console, Bedrock/ Vertex/Foundry	ChatGPT login or API key; cached `~/.codex/auth.json` or credential store	Google login; API key; Vertex AI (ADC/service account/API key); `.gemini/.env`	Google account sign-in (preview: personal Gmail)
Pricing / quotas	Subscription mentioned; numeric limits unspecified	Included with select ChatGPT plans; API-key billed at standard rates; rate-limits “2x” on some plans	Numeric request quotas by auth mode	Preview “no charge” + rate-limits refreshed ~5 hours (excerpt)

Attribute	Claude Code	OpenAI Codex	Gemini CLI	Google Antigravity
Observability	Settings reference includes telemetry env usage in example; full telemetry docs not retrieved	Unspecified	OpenTelemetry support with settings/env vars	Unspecified

Key citations supporting the matrix include: Claude scopes/settings and permissions [106]; Codex install/auth/config/skills/multi-agent [107]; Gemini CLI skills/hooks/tools/sandbox/headless/MCP/pricing [108]; Antigravity mission control/policies/browser/skills excerpts [109].

Mapping of "project instruction context" across platforms

Concept	Claude Code	OpenAI Codex	Gemini CLI	Google Antigravity
Primary persistent instruction file	`CLAUDE.md` and `.claude/rules/*.md`	`AGENTS.md` guide exists (retrieved only as excerpt)	`GEMINI.md` hierarchical memory	"Rules" + workflows panels; file-based scheme unspecified
Hierarchy / scoping	Walks directory tree; subdir CLAUDE.md loads on demand; rules can be path-scoped via frontmatter	Unspecified in retrieved excerpt; AGENTS.md implies override layering	Global → workspace → JIT context via tool access up to trusted root	Unspecified
Imports	`@path` imports with max depth 5	Unspecified	`@file` imports documented	Unspecified

Claude's CLAUDE.md + rules scoping and imports are documented. [13] Gemini's hierarchical system and imports are documented. [110] Codex references an AGENTS.md guide but its full resolution rules weren't retrieved (excerpt indicates layered overrides). [111]

Reference architectures and design patterns for a cross-platform coding agent

Design goal

Build a single "coding agent product" that can be deployed across: - **Claude Code** (plugins/skills/subagents + MCP) - **OpenAI Codex** (skills + config/rules + MCP) - **Gemini CLI** (extensions/skills/hooks + MCP + headless mode) - **Antigravity** (IDE-first orchestration with rules/skills and browser agent)

Given the shared adoption of Agent Skills and MCP, the best architecture is to treat these as **platform-neutral integration planes** and implement surface-specific adapters.

Canonical architecture

```
flowchart TB
  U[Developer] --> S[Surface Adapter<br/>CLI / IDE / Agent Manager]

  S --> O[Orchestrator Core]
  O --> P[Policy Engine<br/>approvals, allow/deny, trust]
  O --> K[Knowledge & Context Loader<br/>project instructions + skill metadata]
  O --> R[Runtime Planner<br/>plan/execute loop]

  R --> SK[Skill Registry<br/>Agent Skills Packs]
  R --> TOOLS[Tool Router]

  TOOLS --> MCP[MCP Client Layer<br/>servers + schemas]
  TOOLS --> LOCAL[Local Tools<br/>fs, shell, git]

  P --> SANDBOX[Sandbox/Isolation Layer]
  SANDBOX --> LOCAL

  O --> OBS[Observability<br/>logs/metrics/traces]
```

This design is an inference drawn from how each platform structures extension and safety: - MCP as the standardized tool plane in Claude Code, Codex, and Gemini CLI. [112]
- Skill packs following Agent Skills in Claude Code, Codex, Gemini CLI. [113]
- Policy interception via permissions (Claude), rules (Codex), and hooks (Gemini). [114]
- Isolation via sandboxing paths (Gemini) and warning-based bypass modes (Claude). [115]

Patterns for skills/workflows/hooks portability

Use Agent Skills as the portability substrate

Agent Skills appear in Claude Code, Codex, and Gemini CLI as `SKILL.md` directories (with optional assets/scripts), and are discoverable and activatable based on short descriptions. [116]

Recommended “portable skill pack” layout (platform-neutral)

```
skills/
  deploy/
    SKILL.md
    templates/
      rollout.md
    scripts/
```

```
      validate.sh
  security_review/
    SKILL.md
    scripts/
      semgrep.sh
```

This aligns with how Claude Code and Gemini CLI describe skills as directories with `SKILL.md` plus supporting assets/scripts. [117] Codex similarly treats a skill as a folder with `SKILL.md`. [48]

Split "rules" vs "skills" explicitly

A cross-platform agent should classify guidance into: - **Always-on constraints** (project policy): Claude `.claude/rules/`, Gemini `GEMINI.md`, Codex `AGENTS.md`/rules. - **On-demand procedures**: skills (SKILL.md) invoked explicitly or implicitly.

This is explicitly recommended by Claude Code docs: rules for persistent scope and skills for task-specific instructions that don't need to be in context all the time. [14] Gemini CLI makes the same conceptual distinction between GEMINI.md and skills as "on-demand expertise." [118]

Implement "hook adapters" as policy enforcement

Gemini hooks provide a particularly strong reference design: - Hook event system (BeforeTool, AfterTool, etc.) - Strict JSON output for deterministic interception behavior - Ability to rewrite/block tool calls [119]

Portable approach: - Author enterprise enforcement as **standalone scripts** that accept a common JSON input and emit a common JSON decision output. - Wrap these scripts differently per platform: - Gemini: native hooks in `settings.json`. [80] - Claude: hooks packaged in plugins/hook config (event names matter; scripts must be executable). [25] - Codex: rules system at argv-level; use it to deny classes of commands as a first line of defense, and (if hooks exist) apply scripts similarly. [51]

Cross-platform MCP connector strategy

Because MCP exists in Claude Code, Codex, and Gemini CLI, treat MCP servers as the canonical "tool backend." [112]

Recommended patterns: - Prefer **stdio MCP servers** for local, auditable tooling (git, static analyzers, DB clients). - Prefer **HTTP/SSE MCP servers** for hosted internal services, with OAuth/token controls where supported (Gemini CLI has deep OAuth support documented). [120] - Always implement tool filtering/allowlisting at the MCP-client level when supported (`includeTools`/`excludeTools` in Gemini CLI). [121] - Never hardcode secrets into config; map env vars through wrappers (Gemini's MCP tutorial demonstrates mapping `GITHUB_PERSONAL_ACCESS_TOKEN` via `${...}` rather than literal tokens in config). [85]

Implementation checklists and migration playbooks

Implementation checklist for a coding-focused agent workflow

Use this as a “platform-agnostic” readiness checklist, then apply platform-specific steps.

- [ ] Define a **project policy file** (coding standards, build/test commands, security rules) in the platform-native form:
- Claude: `./.claude/CLAUDE.md` + `.claude/rules/*.md` with path scoping if needed. [13]
- Gemini: `GEMINI.md` with imports for modularity. [68]
- Codex: AGENTS.md layering (guide exists; details partially excerpted). [111]
- [ ] Create a **skill library** following Agent Skills (`skills/<name>/SKILL.md`), with short descriptions that include both “should” and “should not” trigger conditions (explicitly recommended in Codex skills excerpt). [122]
- [ ] Add a **sandbox strategy**:
- Gemini: enable via `-s`, `GEMINI_SANDBOX`, or settings; choose backend (docker/seatbelt/runsc/lxc). [123]
- Claude: apply permissions rules and avoid bypassPermissions except in isolated environments (explicit warning). [32]
- [ ] Implement **tool governance**:
- Deny access to `.env`, secrets directories, and dangerous shell patterns (Claude permission rules shown; Gemini supports exclude tool patterns; Codex supports pattern-based argv rules). [124]
- [ ] Add MCP connectors for core integrations (GitHub, issue trackers, internal docs).
- Claude: `.mcp.json` project configuration with approval gating. [28]
- Gemini: `mcpServers` in settings, environment sanitization, OAuth where needed. [120]
- Codex: MCP supported in CLI/IDE extension (config specifics not retrieved). [52]
- [ ] Add lifecycle interception:
- Gemini hooks for `BeforeTool` and `AfterModel` to block exfiltration patterns and redact outputs. [119]
- Claude hooks via plugin packaging (ensure correct event names and executable scripts). [25]
- [ ] Enable observability:
- Gemini: OpenTelemetry settings and ensure prompts logging is configured appropriately (`logPrompts`). [96]
- Claude: settings examples show telemetry environment variables (full telemetry doc not retrieved here). [33]
- [ ] Establish authentication strategy:
- Prefer API keys for automation where supported (Gemini and Codex both document API key modes; Codex explicitly recommends API keys as default for automation). [125]
- For enterprise: consider cloud-provider auth modes (Claude supports Bedrock/Vertex/Foundry). [30]

Migration considerations between platforms

Migrating “project rules” and persistent instructions

- Claude Code’s CLAUDE.md + `.claude/rules/` maps conceptually to Gemini CLI’s `GEMINI.md` plus its own hierarchical and just-in-time context discovery. [126]

- Codex's `AGENTS.md` guidance suggests a similar override layering model ("global file first, repository root second, payments override last"), although the full algorithm wasn't retrieved. [111]

Recommendation: Maintain a canonical `policy/` directory in-repo, then generate platform-native files: - `.claude/CLAUDE.md` imports `@policy/core.md` - `GEMINI.md` imports `@./policy/core.md` - `AGENTS.md` (Codex) imports or inlines the same content (format specifics unspecified)

Migrating reusable commands and workflows

- Claude Code merges custom commands into skills, where both `.claude/commands/deploy.md` and `.claude/skills/deploy/SKILL.md` create `/deploy`. [15]
- Gemini CLI uses `.toml` custom commands in `.gemini/commands/` and supports embedding shell output via `!{...}` and file content via `@{...}`. [127]
- Codex uses skills (and likely slash commands) but the full custom command format beyond skill invocation was not retrieved.

Recommendation: Encode complex workflows as skills, not slash commands, whenever portability matters.

Migrating hook-based enforcement

Gemini's hook system is the most formally specified (events, schemas, strict JSON). [119] The most portable enforcement approach is: - Implement controls as external scripts with stable JSON I/O. - Use platform adapters to translate lifecycle events: - Gemini: direct event mapping. - Claude: approximate mapping via tool-use events (e.g., PostToolUse). - Codex: use "Rules" for pre-execution blocking with argv patterning; where deeper interception is needed, add MCP wrappers.

Migrating tool integration (MCP)

Because Claude Code and Gemini CLI store MCP server definitions differently (`.mcp.json` vs `settings.json`), implement an internal "connector registry" and generate platform-specific configuration: - `.mcp.json` for Claude projects [28] - `mcpServers` map under `~/.gemini/settings.json` or project `.gemini/settings.json` [128] - Codex MCP config format unspecified, but MCP is supported in CLI/IDE extension. [52]

Security, privacy, and governance implications

Threat model themes shared across platforms

All four platforms aim to automate real developer actions (editing files, running commands, calling external services). This creates a common set of risks:

- **Code and secret exfiltration** through tool outputs, web fetches, or malicious prompt injection in repositories.
- **Untrusted project execution** risk when opening repos that contain hooks/plugins/rules that execute code.
- **Credential leakage** through environment variables, plaintext caches, or mis-scoped configs.
- **SSRF / internal network access** via web-fetch tooling.

The docs of Claude Code, Codex, and Gemini CLI address these explicitly through some combination of: - approvals + permission rules, - sandboxing, - trust gating for project-scoped configs, - warnings and explicit "unsafe modes," - and tool filtering / denylisting. [129]

Credential handling and storage

Claude Code - Stores credentials securely (macOS Keychain) and supports multiple auth types; can use `apiKeyHelper` scripts with refresh TTL controls. [31]
Security guidance: prefer `apiKeyHelper` for short-lived keys in enterprise environments, and manage TTL centrally.

OpenAI Codex - Caches login details locally and may store them in plaintext at `~/.codex/auth.json` (or OS credential store). [41]
Security guidance: treat `~/.codex/auth.json` as a secret artifact; enforce strict filesystem permissions; avoid persisting it in ephemeral CI runners; rotate API keys and prefer OS credential store where available.

Gemini CLI - Supports `.gemini/.env` for environment variable loading and warns that exporting secrets in shell startup files exposes them to any process launched from that shell. [130]
- Extensions can mark settings as `sensitive: true` to store keys in keychain and obfuscate output. [131]
Security guidance: centralize secrets in keychain-backed settings whenever possible; avoid committing `.gemini/.env` in repos.

Untrusted project risk: hooks and plugins

Gemini CLI warns that hooks execute arbitrary code with your user privileges, and highlights project-level hooks as particularly risky, with fingerprinting and warnings on changes. [80]

Claude Code's ecosystem includes hooks and plugins, and troubleshooting guidance emphasizes executable scripts, correct hook naming, and plugin structural correctness—implicitly reinforcing that these are code-execution surfaces that must be governed. [24]

Codex's project-scoped config is loaded only when you "trust the project," which is a crucial mitigation pattern for config-based execution and rules. [43]

Governance recommendation:
Adopt a policy that all project-scoped automation (hooks/plugins/MCP servers) must be: - reviewed in code review, - pinned to versions (semantic versioning where relevant), - and blocked in untrusted repos by default.

Tool execution and sandboxing

Gemini CLI provides the most comprehensive sandboxing scheme (Seatbelt, Docker/Podman, runsc, LXC), with explicit precedence order for enabling sandboxing. [123] This is a strong model for enterprise deployment: enforce container-based sandboxing for all mutating tools in CI-like environments.

Claude Code's permissions system includes a `bypassPermissions` mode but explicitly warns it should only be used in isolated environments like containers/VMs; admins can disable it via managed settings. [32] This should be treated as the equivalent of "dangerous override" modes.

Antigravity's onboarding policy options (terminal auto-execution, artifact review frequency, JS execution) demonstrate a similar framing: autonomy knobs directly map to security exposure. [98]

Network access and SSRF-style risks

Gemini's tools reference explicitly warns that `web_fetch` can access local/private network addresses (including localhost) and may pose a security risk under untrusted prompts. [72]

Codex and Claude both support web access patterns (Claude has `WebFetch` permissions and MCP connectors; Codex supports MCP tools that may include browser integrations), but the most explicit SSRF warning in the retrieved sources is Gemini's. [132]

Mitigation pattern:
Implement domain allowlists and private-network blocks at the tool/proxy layer, not just in prompts. Gemini's MCP config supports custom headers and OAuth controls, and Claude's permissions allow domain specifiers for `WebFetch`. [133]

Observability and privacy tradeoffs

Gemini CLI's OpenTelemetry integration includes `logPrompts` (default true) and supports exporting telemetry to local or Google Cloud targets. [96] This is powerful but creates privacy considerations: prompt logging can capture proprietary code and secrets if tool-output masking is incomplete.

Gemini also documents that it logs events such as tool output masking for privacy and hook calls. [97]

Governance recommendation:
- Treat "prompt + tool output logging" as sensitive production telemetry. - Default to redaction and disable prompt logging in regulated environments unless explicitly approved. - Use separate telemetry projects/accounts where supported. - Ensure that audit logs do not become secondary exfiltration channels.

Sample definitions and scripts

A portable Agent Skills SKILL.md template

```
---
name: security-review
description: |
  Trigger when asked to assess security risks in code changes, dependencies, or
configuration.
  Do NOT trigger for general refactoring without security implications.
```

```
---

You are a security reviewer.

Checklist:
- Identify secrets exposure risks (.env, logs, debug output)
- Identify injection risks (shell, SQL, template)
- Identify auth/session handling issues
- Recommend minimal safe changes

When appropriate, run static analysis scripts in ./scripts/ and summarize
results.
```

This structure aligns with how Claude Code and Codex document `name` and `description` frontmatter in `SKILL.md`, and how Gemini CLI treats skill metadata as discoverable triggers. [134]

Gemini CLI hook example: block unsafe shell commands

A Gemini hooks `settings.json` entry can intercept tools like `write_file` or shell execution, using regex matchers and a command hook. [79]

```
{
  "hooks": {
    "BeforeTool": [
      {
        "matcher": "run_shell_command",
        "hooks": [
          {
            "name": "block-rm-rf",
            "type": "command",
            "command": "$GEMINI_PROJECT_DIR/.gemini/hooks/block-dangerous.sh",
            "timeout": 5000
          }
        ]
      }
    ]
  }
}
```

Gemini's hooks require strict JSON-only stdout and recommend stderr for debug logs. [79]

Gemini CLI custom command: generate a commit message from staged diff

Gemini CLI supports `!{...}` command injection and documents a concrete commit-message example. [135]

```
description = "Generates a Git commit message based on staged changes."

prompt = """
Please generate a Conventional Commit message based on the following git diff:

```diff
!{git diff --staged}
```

"""
```

```
### Codex config snippet: enable multi-agent and disable a skill

```toml
[features]
multi_agent = true

[[skills.config]]
path = "/path/to/skills/security-review/SKILL.md"
enabled = false
```
```

Codex documents enabling multi-agent via `[features] multi_agent = true` and disabling skills via `[[skills.config]]`. [136]

Claude `.claude/rules/` path-scoped rule example

Claude Code documents YAML frontmatter `paths:` for rules to scope instructions to matching file patterns. [13]

```
---
paths:
  - "src/api/**/*.ts"
---

# API Development Rules
- All API endpoints must include input validation
- Use the standard error response format
- Include OpenAPI documentation comments
```

Closing synthesis

The three fully accessible doc sets (Claude Code, OpenAI Codex, Gemini CLI) converge on a compelling cross-platform foundation: **Agent Skills for portable expertise** and **MCP for portable tool integrations**,

surrounded by a control plane of **trust gating, permissions/approvals, sandboxing, and hook/rule interception**. [137] Google Antigravity's accessible materials align conceptually with that direction (agent orchestration UI, explicit autonomy/security policies, browser subagent, skills directories), but a full technical mapping is limited by retrievability of the primary docs pages in this research session. [138]

---

[1] [15] [16] [17] [18] [19] [113] [116] [117] [134] [137] https://code.claude.com/docs/en/skills
https://code.claude.com/docs/en/skills

[2] [8] [12] [33] [106] [124] https://code.claude.com/docs/en/settings
https://code.claude.com/docs/en/settings

[3] [36] [40] [41] https://developers.openai.com/codex/auth
https://developers.openai.com/codex/auth

[4] [58] [86] https://geminicli.com/docs/cli/headless/
https://geminicli.com/docs/cli/headless/

[5] [6] [7] [98] [109] [138] https://codelabs.developers.google.com/getting-started-google-antigravity
https://codelabs.developers.google.com/getting-started-google-antigravity

[9] https://code.claude.com/docs/en/features-overview
https://code.claude.com/docs/en/features-overview

[10] [13] [14] [126] https://code.claude.com/docs/en/memory
https://code.claude.com/docs/en/memory

[11] [22] https://code.claude.com/docs/en/agent-teams
https://code.claude.com/docs/en/agent-teams

[20] [21] [23] https://code.claude.com/docs/en/sub-agents
https://code.claude.com/docs/en/sub-agents

[24] [25] [26] https://code.claude.com/docs/en/plugins-reference
https://code.claude.com/docs/en/plugins-reference

[27] [28] [29] [112] https://code.claude.com/docs/en/mcp
https://code.claude.com/docs/en/mcp

[30] [31] https://code.claude.com/docs/en/authentication
https://code.claude.com/docs/en/authentication

[32] [114] [129] [132] [133] https://code.claude.com/docs/en/permissions
https://code.claude.com/docs/en/permissions

[34] [37] [107] https://developers.openai.com/codex/quickstart
https://developers.openai.com/codex/quickstart

[35] [38] https://developers.openai.com/codex/cli
https://developers.openai.com/codex/cli

[39] https://developers.openai.com/codex/ide/features
https://developers.openai.com/codex/ide/features

[42] https://developers.openai.com/codex/cli/reference
https://developers.openai.com/codex/cli/reference

43 45 https://developers.openai.com/codex/config-reference
https://developers.openai.com/codex/config-reference

44 https://developers.openai.com/codex/config-sample
https://developers.openai.com/codex/config-sample

46 47 48 122 https://developers.openai.com/codex/skills
https://developers.openai.com/codex/skills

49 136 https://developers.openai.com/codex/multi-agent
https://developers.openai.com/codex/multi-agent

50 51 https://developers.openai.com/codex/rules
https://developers.openai.com/codex/rules

52 https://developers.openai.com/codex/mcp
https://developers.openai.com/codex/mcp

53 https://developers.openai.com/codex/app-server
https://developers.openai.com/codex/app-server

54 https://developers.openai.com/codex/changelog
https://developers.openai.com/codex/changelog

55 https://developers.openai.com/codex/pricing/
https://developers.openai.com/codex/pricing/

56 https://developers.openai.com/codex/noninteractive
https://developers.openai.com/codex/noninteractive

57 https://geminicli.com/docs/get-started/
https://geminicli.com/docs/get-started/

59 60 61 https://geminicli.com/docs/get-started/installation/
https://geminicli.com/docs/get-started/installation/

62 63 64 65 66 125 130 https://geminicli.com/docs/get-started/authentication/
https://geminicli.com/docs/get-started/authentication/

67 68 110 https://geminicli.com/docs/cli/gemini-md/
https://geminicli.com/docs/cli/gemini-md/

69 70 71 72 73 https://geminicli.com/docs/reference/tools/
https://geminicli.com/docs/reference/tools/

74 75 108 118 https://geminicli.com/docs/cli/skills/
https://geminicli.com/docs/cli/skills/

76 77 78 79 80 119 https://geminicli.com/docs/hooks/
https://geminicli.com/docs/hooks/

81 82 83 84 120 121 128 https://geminicli.com/docs/tools/mcp-server/
https://geminicli.com/docs/tools/mcp-server/

85 https://geminicli.com/docs/cli/tutorials/mcp-setup/
https://geminicli.com/docs/cli/tutorials/mcp-setup/

87 88 115 123 https://geminicli.com/docs/cli/sandbox/
https://geminicli.com/docs/cli/sandbox/

89 90 https://geminicli.com/docs/resources/quota-and-pricing/
https://geminicli.com/docs/resources/quota-and-pricing/

91 92 https://geminicli.com/docs/extensions/
https://geminicli.com/docs/extensions/

93 https://geminicli.com/docs/extensions/reference/
https://geminicli.com/docs/extensions/reference/

94 https://geminicli.com/docs/extensions/writing-extensions/
https://geminicli.com/docs/extensions/writing-extensions/

95 131 https://geminicli.com/docs/extensions/best-practices/
https://geminicli.com/docs/extensions/best-practices/

96 97 https://geminicli.com/docs/cli/telemetry/
https://geminicli.com/docs/cli/telemetry/

99 https://antigravity.google/docs/home
https://antigravity.google/docs/home

100 https://antigravity.google/docs/get-started
https://antigravity.google/docs/get-started

101 https://antigravity.google/docs/agent
https://antigravity.google/docs/agent

102 https://antigravity.google/docs/rules-workflows
https://antigravity.google/docs/rules-workflows

103 https://antigravity.google/docs/browser
https://antigravity.google/docs/browser

104 https://antigravity.google/docs/skills
https://antigravity.google/docs/skills

105 https://antigravity.google/blog/introducing-google-antigravity?id=GoogleAntigravity
https://antigravity.google/blog/introducing-google-antigravity?id=GoogleAntigravity

111 https://developers.openai.com/codex/guides/agents-md
https://developers.openai.com/codex/guides/agents-md

127 135 https://geminicli.com/docs/cli/custom-commands/
https://geminicli.com/docs/cli/custom-commands/